

“Implementation and Demonstration of Argument Passing Mechanisms in Linux Process Execution”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

MARREDDY SAI BHAVITHA REDDY 2520030551

BHIMAVARAPU SAI SARANYA 2520030258

LAKSHMI ASWIKA INTURI 2520030221

Under the Esteemed Guidance of

Course Faculty / Project Guide

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

**K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled “Implementation and Demonstration of Argument Passing Mechanisms in Linux Process Execution” is a record of bonafide work of MARREDDY SAI BHAVITHA REDDY - 2520030551, BHIMAVARAPU SAI SARANYA - 2520030258, LAKSHMI ASWIKI INTURI - 2520030221 submitted in partial fulfillment for the course Operating Systems and Systems Programming (25CS2104E) at K L (Deemed to be) University. The work presented in this report is based on the project implementation and demonstration of argument passing mechanisms in Linux process execution.

The project demonstrates the fork()–exec() cycle, construction and passing of argv[] and envp[] arrays, mini-shell command parsing, parent-child synchronization, and verification through the Linux /proc filesystem. The results and methodology are presented for academic purposes and reflect the scope defined in the project presentation.

**MARREDDY SAI BHAVITHA REDDY 2520030551
BHIMAVARAPU SAI SARANYA 2520030258
LAKSHMI ASWIKI INTURI 2520030221**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the mini project based report entitled “Implementation and Demonstration of Argument Passing Mechanisms in Linux Process Execution” is a bonafide work done and submitted by MARREDDY SAI BHAVITHA REDDY - 2520030551, BHIMAVARAPU SAI SARANYA - 2520030258, LAKSHMI ASWIKA INTURI - 2520030221, in partial fulfillment of the requirements of the course Operating Systems and Systems Programming (25CS2104E), during the academic year 2026-27, Term-I.

The project demonstrates argument passing across the Linux `fork()`–`exec()` process execution cycle using custom `argv[]` and `envp[]` arrays, a mini-shell, helper programs, and the `/proc` filesystem.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We express our sincere thanks to the faculty, department, and university for providing the opportunity, resources, and academic environment required to complete this project.

We express our greatest appreciation to our course faculty and project guide for the guidance, encouragement, and technical support provided throughout the project. Their support helped us understand and demonstrate Linux process creation, program execution, argument passing, and process inspection.

We express our gratitude to the Department of Computer Science Engineering and the university for providing the facilities and learning environment required for this work. We also thank the teaching and non-teaching staff members who supported us directly or indirectly.

Finally, we sincerely thank our teammates, classmates, friends, and family members for their cooperation and encouragement during the preparation, implementation, testing, and presentation of this project.

MARREDDY SAI BHAVITHA REDDY 2520030551

BHIMAVARAPU SAI SARANYA 2520030258

LAKSHMI ASWIKA INTURI 2520030221

Abstract

Argument passing is a fundamental part of Linux process execution because a newly executed program receives command-line parameters and environment information through structured argument and environment arrays. Students often use `argc` and `argv` without understanding how these values reach a newly executed program. This project provides a concrete and testable demonstration of argument passing across the `fork()`–`exec()` cycle.

The project constructs custom `argv` and `envp` arrays and passes them using `execve()/execvp()`. A mini-shell tokenizes user commands and forwards arguments to child processes. Helper programs can print the received `argc`, `argv[]`, and environment information so that the argument-passing mechanism becomes directly observable.

The Linux `/proc` filesystem is used to inspect command-line and environment data through `/proc/[pid]/cmdline` and `/proc/[pid]/environ`. The project therefore connects system calls, process creation, command-line parsing, synchronization, and runtime process inspection in one practical demonstration.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		1–5
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	Background Study and OS Concepts		6–12
		2.1 Linux Process Model	
		2.2 fork() System Call	
		2.3 execve() / execvp()	
		2.4 argc, argv[] and envp[]	
3	System Requirements		13–17
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		18–22
		4.1 C / C++ Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 GDB Debugger	
		4.5 Linux Commands and Utilities	
		4.6 /proc Filesystem	
5	System Architecture and Design		23–29
		5.1 Overall System Architecture	
		5.2 System Workflow	
		5.3 Process Argument Flow	
		5.4 Module Description	
		5.5 Input and Output Design	
6	Implementation		30–45
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Argument and Environment Construction	
		6.4 Linux System Calls / APIs Used	
		6.5 Process Creation and Execution	
		6.6 Mini-Shell / Tokenization	
		6.7 Synchronization and /proc Verification	
		6.8 Helper Program Implementation	
7	Execution and Demonstration		62–67
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	

		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	Conclusion		79–83
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	References		84–86
		9.1 Books	
		9.2 Research Papers	
		9.3 Linux / GNU Documentation	
		9.4 Online Resources	

CHAPTER -1 INTRODUCTION

CHAPTER -2 BACKGROUND STUDY AND OS CONCEPTS

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Linux process execution involves creating and executing processes while passing the information required by the new program. The project focuses on making the normally hidden argument-passing mechanism observable. It demonstrates how a command entered by a user becomes `argv[]` and `envp[]` data for a newly executed program.

The central process flow is based on `fork()` and `execve()/execvp()`. `fork()` creates a child process, while `execve()/execvp()` replaces the child process image with the selected program and supplies its arguments and environment information.

1.2 Problem Statement

Students often use `argc` and `argv` without understanding how these values reach a program. The internal process creation flow involving `fork()`, `exec()`, command-line parsing, and the process execution environment is frequently treated as a black box. This project provides a concrete and testable demonstration of argument passing across Linux processes.

1.3 Scope of the Project

- Demonstrate command-line argument passing during the `fork()`–`exec()` cycle.
- Construct and pass custom `argv[]` and `envp[]` arrays.
- Build a mini-shell that tokenizes user commands.
- Support multiple and quoted arguments.
- Use helper programs to display received `argc`, `argv[]`, and environment information.
- Inspect `/proc/[pid]/cmdline` and `/proc/[pid]/environ` for verification.

1.4 Objectives of the Project

- Construct and pass custom `argv/envp` arrays using `fork()` and `execve()/execvp()`.
- Build a mini-shell that handles multiple and quoted arguments.
- Inspect argument and environment data using the `/proc` filesystem.
- Explain how the kernel and C runtime provide `argc`, `argv[]`, and `envp[]` to a new process.

1.5 Importance of the Application

The project converts an abstract operating-system concept into an observable experiment. It helps students connect process creation, program execution, command-line parsing, and runtime process information.

1.6 Target Users / Audience

- Operating Systems students
- Systems Programming students
- Students learning Linux system calls
- Faculty demonstrating process execution concepts

CHAPTER -2 BACKGROUND STUDY AND OS CONCEPTS

2.1 Linux Process Model

A Linux process is an executing instance of a program. The project studies the relationship between a parent process and a child process, followed by replacement of the child's program image.

2.2 fork() System Call

fork() creates a child process from the calling process. In this project it establishes the parent-child relationship required before executing the target program in the child.

2.3 execve() / execvp()

The exec family replaces the current process image with a new program image. execve() accepts explicit argument and environment arrays, while execvp() executes a program using the PATH search mechanism.

2.4 argc, argv[] and envp[]

argc represents the argument count, argv[] contains command-line argument strings, and envp[] represents environment information supplied to the executed program.

2.5 Parent-Child Synchronization

wait() and waitpid() are used to synchronize parent and child execution so that the parent can account for the child process after launching the target program.

2.6 /proc Filesystem

The Linux /proc filesystem provides runtime process information. The project uses /proc/[pid]/cmdline to inspect command-line data and /proc/[pid]/environ to inspect environment information.

CHAPTER -3 SYSTEM REQUIREMENTS

3.1 Hardware Requirements

A modern 64-bit computer with a standard keyboard, mouse, display, and sufficient RAM and storage for Linux development is adequate.

3.2 Software Requirements

Linux/Ubuntu, GCC, GDB, a C/C++ development environment, and access to the /proc filesystem are required.

3.3 Operating System Environment – Ubuntu / WSL

The project is designed for Linux/Ubuntu. Ubuntu may be used directly or through a suitable Linux environment such as WSL when configured for Linux development.

3.4 Development Tools and Compilers

GCC is used for compilation, GDB for debugging, and the Linux terminal and /proc filesystem for execution and runtime inspection.

CHAPTER -4 TECHNOLOGY AND TOOLS USED

4.1 C / C++ Programming

The project uses C/C++ programming concepts, with the core operating-system demonstration centered on Linux process APIs and process-related data structures.

4.2 GCC Compiler

GCC compiles the C source files into executable programs that can be run in the Linux environment.

4.3 Linux System Calls

The project uses `fork()`, `execve()`, `execvp()`, `wait()`, and `waitpid()` to demonstrate process creation, program replacement, and synchronization.

4.4 GDB Debugger

GDB is used to debug process creation and argument-passing logic.

4.5 Linux Commands and Utilities

The Linux terminal and standard utilities provide the execution environment for compiling, running, and inspecting the project.

4.6 /proc Filesystem

`/proc/[pid]/cmdline` and `/proc/[pid]/environ` are used for runtime inspection and verification of process arguments and environment data.

CHAPTER -5 SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

User Command → Mini-Shell / Parent Process → Tokenization → argv[] and envp[] → fork() → Child Process → execve()/execvp() → New Program → /proc Verification

5.2 System Workflow

- The user enters a command.
- The mini-shell tokenizes the command into the program and its arguments.
- argv[] and envp[] are constructed or forwarded.
- fork() creates a child process.
- The child calls execve()/execvp().
- The new program receives argc, argv[], and envp[].
- The parent synchronizes using wait()/waitpid().
- The process is verified using /proc.

5.3 Process Argument Flow

fork() → exec() → New Program

User Command → Mini-Shell / Parent → Tokenization → argv[] and envp[] → fork() → execve()/execvp() → argc, argv[], envp[] → /proc verification

5.4 Module Description

- Command Input Module – accepts the user command.
- Tokenizer – separates command and arguments, including quoted input.
- Argument Builder – constructs argv[].
- Environment Builder – constructs or forwards envp[].
- Process Manager – creates and manages the child process.
- Execution Module – executes the selected program.
- Helper Program – displays received argc, argv[], and environment data.
- Verification Module – inspects runtime information through /proc.

5.5 Input and Output Design

Input consists of commands with zero or more arguments. Output displays the received argument count and argument strings, with environment information available when environment propagation is demonstrated. /proc inspection provides an additional verification path.

CHAPTER -6 IMPLEMENTATION

6.1 Module-wise Implementation

The implementation is divided into command input, tokenization, argument/environment construction, process creation, execution, helper-program output, synchronization, and /proc verification.

6.2 Algorithm / Pseudocode

START → Read command → Tokenize input → Construct argv[] and envp[] → fork() → IF child THEN execve()/execvp() ELSE wait()/waitpid() → Verify through /proc → Repeat → END.

6.3 Argument and Environment Construction

The command and its arguments are organized into argv[]. Environment information is represented through envp[]. These arrays are passed to the execution interface.

6.4 Linux System Calls / APIs Used

fork() creates the child; execve()/execvp() replaces the process image; wait()/waitpid() synchronize execution; getopt() may be used for structured option parsing.

6.5 Process Creation and Execution

The parent calls fork(). The child follows the execution path and calls execve()/execvp(). A successful execve replaces the child program image with the requested program.

6.6 Mini-Shell / Tokenization

The mini-shell converts human-readable command input into structured arguments. Multiple arguments and quoted multi-word arguments are handled as required by the project scope.

6.7 Synchronization and /proc Verification

The parent uses wait()/waitpid() to synchronize with the child. Runtime information is verified through /proc/[pid]/cmdline and /proc/[pid]/environ.

6.8 Helper Program Implementation

A helper program displays argc, argv[], and environment information so the effect of argument passing can be directly observed.

CHAPTER -7 EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

A typical C source file can be compiled using GCC with: `gcc source.c -o program`

7.2 Execution Commands

Start the mini-shell or parent executable, enter commands with no arguments, multiple arguments, and quoted arguments, then inspect the corresponding process information.

7.3 Sample Inputs and Outputs

Test cases include no arguments, a single argument, multiple arguments, a quoted multi-word argument, environment propagation, and `/proc` verification. The helper program should display `argc` and each `argv[]` element.

7.4 Screenshots of Execution

Screenshots of the actual Ubuntu terminal execution can be inserted here without changing the report structure. They should show compilation, mini-shell execution, argument output, and `/proc` verification.

7.5 Demonstration of OS Concepts

The demonstration shows process creation using `fork()`, process-image replacement using `execve()/execvp()`, argument passing through `argv[]`, environment passing through `envp[]`, synchronization using `wait()/waitpid()`, and runtime inspection using `/proc`.

CHAPTER -8 CONCLUSION

8.1 Summary of the Project

The project provides a practical demonstration of how command-line arguments and environment information reach a newly executed Linux program through the fork()–exec() cycle.

8.2 Objectives Achieved

The project covers construction and passing of argv[] and envp[], mini-shell tokenization, quoted and multiple arguments, and /proc-based verification.

8.3 Results Achieved

The expected outcome is a working set of Linux command-line utilities and a mini-shell with correct argument and environment passing and verification through program output and /proc.

8.4 Skills and Learning Outcomes

The project strengthens understanding of Linux process creation, exec(), argc/argv/envp, synchronization, command-line parsing, GCC compilation, GDB debugging, and /proc runtime inspection.

CHAPTER -9 REFERENCES

9.1 Books

Operating Systems and Systems Programming course materials and standard Linux process-management references.

9.2 Research Papers

Academic references related to Linux process execution and argument passing may be included by the project team if used in the implementation.

9.3 Linux / GNU Documentation

Linux manual pages for `fork()`, `execve()`, `execvp()`, `wait()`, `waitpid()`, `getopt()`, and `proc` filesystem interfaces; GNU GCC and GDB documentation.

9.4 Online Resources

Official Linux/GNU documentation and course-provided learning resources used during development.